

UNIVERSITY OF DHAKA

Department of Computer Science and Engineering

Faculty of Engineering and Technology (FoET) | Second Science Complex, Dhaka 1000,
BangladeshCSE 2206 — Microcontroller and Embedded System | 2nd Year, 2nd Semester
2024

Laboratory Assignment — Lab 02

Timers, Delay Generation, Duration Measurement
and PWM with WS2812B RGB LED

Course: CSE 2206

Marks: 100

Platform: STM32F446RE Nucleo-64

Issue Date: _____

IDE: STM32CubeIDE 1.14+

Due Date: _____

Prerequisite: Lab-01 (GPIO, UART)

Group Size: _____

Student Name(s): ID: Section / Group:

Lab Partner (if any): ID:

General Instructions — Read Before You Begin

1. This is an **individual or paired** assignment. All submitted code must be your own work; plagiarism will result in a mark of zero for all parties involved.
2. Each task must be implemented **twice**: (i) **Bare-Metal (No-HAL)**: direct register access using CMSIS structs (`TIMx->PSC`, `GPIOA->MODER`, etc.), and (ii) **HAL/CubeMX**: STM32CubeIDE peripheral-generation wizard with `HAL_` API calls.
3. **No code is provided in this document**. Algorithm pseudocode and step-by-step guidance are given; you must translate them into compilable, working C.
4. The **UART2 test harness from Lab-01** (`USART2_SendString()` or `HAL_UART_Transmit()`) is the *only* required output channel. All results must be printed to the host serial terminal (115200 baud, 8N1) so that your demonstrator can verify correctness without an oscilloscope.
5. Use an oscilloscope or logic analyser for post-lab waveform screenshots where indicated.
6. Register your group with the lab instructor before beginning. Sign the bench log at the start and end of each session.

OBE Alignment and Marks Distribution

CO	Outcome Description	PO	Bloom	Tasks	Marks
CO1	Apply timer architecture knowledge: bus assignment, PSC/ARR calculation, timer families, PWM theory	PO1	C3 Apply	T1, T2	25
CO2	Develop and analyse firmware for delay generation, code profiling, PWM, and WS2812B colour control; integrate with Lab-01 UART harness	PO2	C4 Analyse	T3–T6	55
CO3	Evaluate PWM-based embedded systems in real-world contexts (IoT, motor control, RGB lighting, energy efficiency aligned with SDG Goal 9)	PO6	A4 Organise	T7	20
Total				7 Tasks	100

1. Background: STM32F4 Timer Architecture

1.1 Timer Families on the STM32F446

The STM32F446 provides four categories of timer peripherals. Understanding which family a timer belongs to determines which features are available and which bus clock feeds it.

Family	Timers	Bus	Width	Key Features
Advanced	TIM1, TIM8	APB2	16-bit	Complementary outputs, dead-time insertion, break input, MOE bit
General-Purpose (32-bit)	TIM2, TIM5	APB1	32-bit	Input capture, output compare, encoder interface
General-Purpose (16-bit)	TIM3, TIM4	APB1	16-bit	Input capture, output compare, PWM generation
Basic	TIM6, TIM7	APB1	16-bit	Up-counter only, update interrupt, DAC trigger

Table 1: STM32F446 timer families

Timer Clock Frequencies (Reference Configuration)

When the PLL is configured for $f_{\text{SYSCLK}} = 180 \text{ MHz}$ with APB1 prescaler = 4:

$$f_{\text{APB1}} = 45 \text{ MHz}, \quad f_{\text{TIM_CLK (APB1)}} = 2 \times f_{\text{APB1}} = \mathbf{90 \text{ MHz}}$$

For APB2 timers (TIM1, TIM8) with APB2 prescaler = 2:

$$f_{\text{APB2}} = 90 \text{ MHz}, \quad f_{\text{TIM_CLK (APB2)}} = 2 \times f_{\text{APB2}} = \mathbf{180 \text{ MHz}}$$

All calculations in this assignment use these values unless stated otherwise.

1.2 Fundamental Timer Equations

Period and Frequency

$$f_{\text{timer}} = \frac{f_{\text{TIM_CLK}}}{PSC + 1} \quad T_{\text{period}} = \frac{ARR + 1}{f_{\text{timer}}} = \frac{(ARR + 1)(PSC + 1)}{f_{\text{TIM_CLK}}} \quad (1)$$

PWM Duty Cycle

$$D [\%] = \frac{CCR_x}{ARR + 1} \times 100 \quad CCR_x = \frac{D}{100} \times (ARR + 1) \quad (2)$$

DWT Cycle-to-Time Conversion

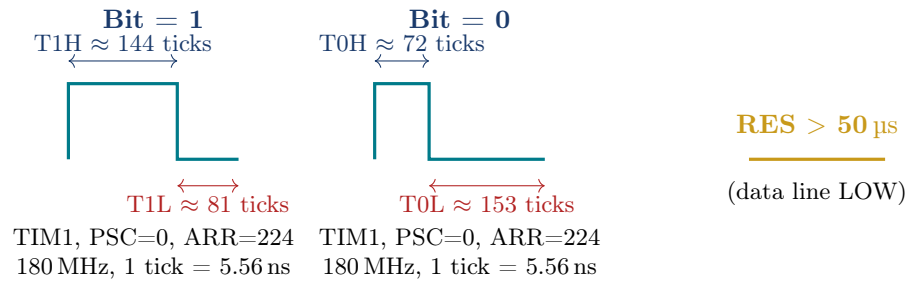
$$t [\mu\text{s}] = \frac{\text{cycles}}{f_{\text{CPU}} [\text{MHz}]} \quad t [\text{ns}] = \frac{\text{cycles} \times 1000}{f_{\text{CPU}} [\text{MHz}]} \quad (3)$$

1.3 WS2812B Single-Wire Protocol Summary

Symbol	Nominal	Tolerance	Meaning
T0H	400 ns	± 150 ns	High time for a logic 0 bit
T0L	850 ns	± 150 ns	Low time for a logic 0 bit
T1H	800 ns	± 150 ns	High time for a logic 1 bit
T1L	450 ns	± 150 ns	Low time for a logic 1 bit
RES	>50 μ s	—	Reset latch: data line held LOW after last bit

Table 2: WS2812B bit-timing specifications

The WS2812B data frame is **24 bits per LED** in **GRB order** (Green MSB first, then Red, then Blue). Multiple LEDs are chained: the first 24 bits are consumed by LED 1, the next 24 bits by LED 2, and so on.

Figure 1: WS2812B bit encoding using TIM1 one-pulse mode at $f_{\text{TIM1}} = 180$ MHz

2. Task 1 — Timer Register Calculations and Configuration

Marks: 10 | CO1/PO1 | Bloom C3: Apply

Pre-lab theory worksheet (submitted before lab session)	5 marks
In-lab viva: explain register writes to demonstrator	5 marks

2.1 Pre-Lab Worksheet

Complete the following calculations **by hand** and enter them in your lab notebook. Show every substitution into the formulae given in Section 1.2 (Equations 1–2).

- Using $f_{\text{TIM6_CLK}} = 90 \text{ MHz}$, determine PSC and ARR values that produce the following TIM6 overflow periods. For each target, show at least two valid (PSC, ARR) pairs and state which you would choose and why.

Target Period	PSC	ARR	Actual Period
1 ms			
100 μs			
500 ms			
1 s			

- For TIM3 generating 1 kHz PWM on APB1 ($f_{\text{TIM3}} = 90 \text{ MHz}$):
 - Determine PSC and ARR.
 - Compute the CCR1 value required for duty cycles of 25 %, 50 %, 75 %, and 100 %.
 - Compute the CCR1 value for an arbitrary duty cycle D in terms of ARR.
- For TIM1 driving a WS2812B LED at $f_{\text{TIM1}} = 180 \text{ MHz}$, $\text{PSC} = 0$:
 - Calculate ARR so that one bit period equals 1250 ns.
 - Calculate CCR1 for a logic-1 bit ($\text{T1H} \approx 800 \text{ ns}$).
 - Calculate CCR1 for a logic-0 bit ($\text{T0H} \approx 400 \text{ ns}$).
 - Verify that your T1H and T0L values are within the WS2812B tolerance given in Table 2.
- The DWT cycle counter runs at $f_{\text{CPU}} = 180 \text{ MHz}$. Convert the following cycle counts to ns, μs , and ms: 3600, 90 000, 9 000 000.

Deliverable T1

Completed pre-lab worksheet signed by your instructor before the hands-on session begins. Incorrect or missing worksheets forfeit the pre-lab marks.

3. Task 2 — Delay Generation: ms, s, Hours

Marks: 15 | CO1-CO2 / PO1-PO2 | Bloom C3-C4

Bare-metal implementation: working	5 marks
UART-verified demo	
HAL/CubeMX implementation: working	5 marks
UART-verified demo	
Post-lab questions (Task 2 section of report)	5 marks

3.1 Overview

Using **TIM6** (Basic Timer on APB1), implement a suite of precise blocking delay functions. Results are verified by printing a timestamp message *before* and *after* each delay call over UART2 (inherited from Lab-01). No oscilloscope is required for this task.

UART Test Harness Requirement

Every delay function must be wrapped in a UART reporting sequence:

1. Print “Starting <N> ms delay...” before calling the delay.
2. Call the delay function.
3. Print “Done.” immediately after the function returns.

The observer times the interval with a stopwatch for verification.

3.2 Algorithm: TIM6 Initialisation

Algorithm A2.1 — TIM6 Initialisation (1 MHz tick = 1 μ s resolution)

1. Enable the TIM6 peripheral clock via the RCC APB1 register.
2. Write a short NOP delay to allow the clock to stabilise.
3. Disable the counter (clear the CEN bit in CR1) before writing configuration registers.
4. Write the Prescaler register (PSC) with the value computed in Task 1 to achieve a 1 MHz timer tick frequency.
5. Write the maximum value into the Auto-Reload Register (ARR) to allow the longest single-shot delay.
6. Set the Update Generation bit (UG) in EGR to force an immediate register reload event.
7. Clear all flags in the Status Register (SR).
8. Start the counter by setting CEN in CR1.

3.3 Algorithm: Delay Functions

Algorithm A2.2 — delay_us(us): Microsecond Delay

1. Reset the counter register (CNT) to zero.
2. Enter a busy-wait loop.
3. Continue looping while the current CNT value is strictly less than the requested number of microseconds **us**.

4. Return when CNT reaches or exceeds `us`.

Constraint: Maximum single call is 65 535 μ s (16-bit counter limit). Consider what happens if an interrupt fires during the wait and how you would detect this.

Algorithm A2.3 — `delay_ms(ms)`: Millisecond Delay

1. Accept a 32-bit unsigned integer `ms`.
2. Repeat `ms` times:
 - a. Call `delay_us(1000)`.

Discussion: Why is a loop around `delay_us(1000)` more robust than a single call to `delay_us(ms * 1000)`? Address counter overflow in your report.

Algorithm A2.4 — `delay_s(sec)`: Second Delay

1. Accept a 32-bit unsigned integer `sec`.
2. Repeat `sec` times:
 - a. Call `delay_ms(1000)`.

Algorithm A2.5 — `delay_hms(h, m, s)`: Hour/Minute/Second Delay

1. Accept three 8-bit unsigned integers: hours `h`, minutes `m`, seconds `s`.
2. Compute the total number of whole seconds:

$$\text{total_s} = h \times 3600 + m \times 60 + s$$
 Use a 32-bit accumulator to avoid overflow.
3. Call `delay_s(total_s)`.

3.4 Demonstration Requirements

Your working program must, in sequence, execute and report over UART:

1. `delay_ms(500)` — print before and after
2. `delay_ms(1000)` — print before and after
3. Toggle the on-board LED (LD2, PA5) ten times at 250 ms on / 250 ms off using your delay functions; print each toggle event
4. `delay_s(3)` — print before and after
5. `delay_hms(0, 0, 5)` — print before and after

3.5 Post-Lab Questions — Task 2

1. If a UART ISR fires and takes 200 μ s to execute during a `delay_ms(1)` call, by how much is the delay extended? Propose a modification to `delay_us()` that would make it interrupt-safe (hint: absolute timestamp instead of relative count).
2. Compare the accuracy of your TIM6 delay with `HAL_Delay()` (SysTick, 1 ms resolution) for delays of 50 μ s, 1 ms, and 1 s. Which method is better for each and why?
3. Calculate the maximum delay achievable by a single call to `delay_hms()` without arithmetic overflow for both unsigned 8-bit and unsigned 32-bit accumulator types.

4. Task 3 — Duration Measurement and Code Profiling

Marks: 15 | CO2 / PO2 | Bloom C4: Analyse

Bare-metal DWT implementation with correct unit conversions	5 marks
HAL implementation: both DWT and TIM2 methods demonstrated	5 marks
Post-lab analysis questions answered with measured data	5 marks

4.1 Overview

You will profile the execution time of code blocks using two independent methods and cross-validate the results. All measurements are reported over UART.

- **Method A — DWT Cycle Counter:** ARM Cortex-M4 hardware counter, increments every CPU clock cycle. Resolution: $1/180\text{ MHz} = 5.56\text{ ns}$.
- **Method B — TIM2 Free-Running Counter:** 32-bit general-purpose timer configured as a $1\text{ }\mu\text{s}$ stopwatch. Suitable for durations up to ≈ 71 minutes.

4.2 Algorithm: DWT Initialisation

Algorithm A3.1 — DWT Cycle Counter Enable

1. Set the TRCENA (Trace Enable) bit in `CoreDebug->DEMCR`.
2. Reset `DWT->CYCCNT` to zero.
3. Set the CYCCNTENA bit in `DWT->CTRL` to start counting.

Note: DWT must be explicitly enabled; it is not active after reset.

4.3 Algorithm: TIM2 Free-Running Initialisation

Algorithm A3.2 — TIM2 Free-Running $1\text{ }\mu\text{s}$ Counter

1. Enable TIM2 clock via RCC APB1 register.
2. Disable the counter (CR1: clear CEN).
3. Set PSC using the value calculated in Task 1 for a $1\text{ }\mu\text{s}$ tick from $f_{\text{TIM2}} = 90\text{ MHz}$.
4. Set ARR to its maximum 32-bit value (`0xFFFFFFFF`) for maximum range.
5. Force register reload by writing UG into EGR.
6. Start the counter (CR1: set CEN).

4.4 Algorithm: Profiling a Code Block

Algorithm A3.3 — Profiling with DWT (Generic Template)

1. Record the current DWT counter value as t_0 .
2. Execute the code block under test.
3. Record the current DWT counter value as t_1 .

4. Compute elapsed cycles: $\Delta = t_1 - t_0$.
(Unsigned subtraction is overflow-safe for durations up to ≈ 23.9 s.)
5. Convert Δ to nanoseconds, microseconds, milliseconds, and seconds using Equation 3.
6. Format a report string and transmit over UART2:
Include: description of the block, cycle count, and all four time-unit representations.

4.5 Code Blocks to Profile

You must profile each of the following and print the results table over UART. Implement a consistent header and row format (see deliverable).

1. A bubble-sort algorithm applied to an integer array of $N = 100$ elements, filled in *reverse sorted order* (worst case). You must implement the sort yourself.
2. The `delay_ms(100)` function from Task 2 — compare measured duration with the expected 100 ms.
3. The `USART2_SendString()` function from Lab-01, transmitting a fixed 48-character string. Use your measurement to calculate the effective baud rate and compare with the configured 115 200 baud.
4. A software integer square-root function (Newton–Raphson or binary search) applied to 1000 successive inputs.
5. A memory copy (byte-by-byte) of 512 bytes from one RAM buffer to another.

Deliverable T3 — UART Output Table

Your program must produce a formatted table over UART with the following columns and rows (values filled at runtime):

#	Block Description	Cycles	ns	us	ms
[1]	Bubble sort N=100	...	...	...	...
[2]	delay_ms(100)	...	...	...	...
[3]	SendString 48B	...	...	...	...

Both DWT and TIM2 values must appear for each block. Justify any differences between the two methods.

4.6 Post-Lab Questions — Task 3

1. The DWT counter overflows at 2^{32} cycles. At $f_{\text{CPU}} = 180$ MHz, what is the maximum measurable duration before overflow? How would you extend this range using a software overflow counter and the DWT overflow interrupt?
2. Profile the *exact same* bubble-sort block three times in succession without re-filling the array. Explain the dramatic decrease in cycle count from the first to subsequent runs. What does this reveal about the algorithm's behaviour on sorted input?
3. Your measured `SendString` duration may exceed the theoretical ms based purely on bit-time calculations. Identify and describe at least two sources of overhead that contribute to the discrepancy.

5. Task 4 — PWM Generation and LED Brightness Control

Marks: 15 | CO1-CO2 / PO1-PO2 | Bloom C4: Analyse

Bare-metal TIM3 PWM: correct waveform and UART duty-cycle table	5 marks
HAL/CubeMX TIM3 PWM: duty cycle sweep and breathing effect	5 marks
Post-lab questions with oscilloscope screenshot	5 marks

5.1 Overview

Configure **TIM3 Channel 1** in **PWM Mode 1** on pin **PA6** (Alternate Function 2) to generate a 1 kHz PWM signal. Control an external LED connected via a $220\ \Omega$ current-limiting resistor (or use the built-in PWM test connector on your lab bench).

5.2 Pin Configuration Summary

MCU Pin	AF	Function	Nucleo Header	Connection
PA6	AF2	TIM3_CH1	CN10 pin 13	External LED + $220\ \Omega$ to GND
PA2	AF7	USART2_TX	CN10 pin 35	ST-LINK VCP (Lab-01)
PA5	GPIO OUT	LD2	On-board	Timing reference toggle

5.3 Algorithm: TIM3 PWM Initialisation

Algorithm A4.1 — TIM3 CH1 PWM Initialisation on PA6

1. Enable peripheral clocks for GPIOA and TIM3 via the RCC registers.
2. Configure PA6 as Alternate Function mode (MODER = 10), Push-Pull output type (OTYPER = 0), Very High Speed (OSPEEDR = 11), and No Pull (PUPDR = 00).
3. Set the Alternate Function register (AFR[0]) for PA6 to AF2 (value 0010 in the 4-bit field for pin 6).
4. Disable TIM3 counter before configuration.
5. Write PSC and ARR values (from Task 1) for a 1 kHz PWM frequency.
6. Configure TIM3 Channel 1 Output Compare mode:
 - a. Set OC1M bits in CCMR1 to PWM Mode 1 (binary 110).
 - b. Set OC1PE (Output Compare 1 Preload Enable) in CCMR1.
7. Enable Channel 1 output (CC1E bit in CCER); set polarity active-high (CC1P = 0).
8. Enable Auto-Reload Preload (ARPE bit in CR1).
9. Force an Update Event (UG in EGR) to load PSC/ARR immediately.
10. Start the counter (CEN in CR1).

Note: For TIM1 (Advanced Timer), you must also set the **MOE** (Main Output Enable) bit in **BDTR**; TIM3 does not require this.

5.4 Algorithm: Duty-Cycle Control

Algorithm A4.2 — PWM_SetDuty(percent): Runtime Duty Cycle

1. Accept an unsigned 8-bit percentage value `pct` (range 0–100).
2. Clamp `pct` to 100 if it exceeds 100.
3. Compute the new CCR1 value using Equation 2:

$$CCR1 = \lfloor pct \times (ARR + 1) / 100 \rfloor$$
4. Write the computed value to TIM3's CCR1 register.

Key property: Because preload is enabled (OC1PE), the new CCR1 value takes effect at the *next* Update Event, ensuring glitch-free transitions.

5.5 Algorithm: Sine-Wave Breathing Effect

Algorithm A4.3 — Breathing Effect Using a Lookup Table

1. Precompute or hard-code a 256-entry lookup table of duty-cycle values (0–100) that follow one complete sine cycle (0 to 360 degrees), offset and scaled so that the minimum is 0 % and the maximum is 100 %.

$$LUT[i] = \left\lfloor 50 \times \left(1 + \sin\left(\frac{2\pi i}{256}\right) \right) \right\rfloor, \quad i = 0, 1, \dots, 255$$

2. Repeat for the desired number of breath cycles:
 - a. For each index i from 0 to 255: call `PWM_SetDuty(LUT[i])`, then call `delay_ms(8)`.
 Total breath period = $256 \times 8 = 2048 \text{ ms} \approx 2 \text{ s}$.

5.6 Demonstration Requirements

Your working program must execute and report over UART:

1. A duty-cycle sweep: 0 %, 10 %, 20 %, ..., 100 %, holding each step for 300 ms. Print a row `Duty = XX% CCR1 = NNN` for each step.
2. Five complete breathing cycles using the sine LUT.
3. A final hold at exactly 50 % with the CCR1 value printed.

5.7 Post-Lab Questions — Task 4

1. What is the effect of removing the Output Compare Preload enable (OC1PE) bit? Under what race condition could this cause a visible glitch on the output waveform?
2. Use an oscilloscope to capture the PWM waveform at 25 %, 50 %, and 75 % duty. Measure the actual frequency and duty cycle from the oscilloscope and compare with the configured values. Include screenshots in your report.
3. Human vision perceives LED brightness logarithmically. A sine-wave LUT does not account for this gamma effect. Propose an alternative LUT function (square-root or power law) and describe qualitatively how it would improve the perceived smoothness of the breathing effect.

6. Task 5 — WS2812B RGB LED: Colour Mixing and Animation

Marks: 20 | CO2–CO3 / PO2, PO6 | Bloom C4–A4

Bare-metal: correct single-LED colour from palette demo	5 marks
Bare-metal: hue sweep / colour animation, UART reporting	5 marks
HAL/CubeMX implementation: same demonstrations	5 marks
Post-lab questions with colour analysis	5 marks

6.1 Overview

Drive a **single WS2812B** (or a chain of up to 8 LEDs) connected to **PA8** using TIM1 Channel 1 (Alternate Function 1) in one-pulse mode. Use the bit-timing values calculated in Task 1.

6.2 Colour Mixing Reference

Colour Name	R	G	B	GRB Wire [hex]	Notes
Red	255	0	0	0x00 0xFF 0x00	Pure red channel
Green	0	255	0	0xFF 0x00 0x00	Pure green channel
Blue	0	0	255	0x00 0x00 0xFF	Pure blue channel
Yellow	255	255	0	0xFF 0xFF 0x00	R + G additive mix
Cyan	0	255	255	0xFF 0x00 0xFF	G + B additive mix
Magenta	255	0	255	0x00 0xFF 0xFF	R + B additive mix
White	255	255	255	0xFF 0xFF 0xFF	All channels full
Warm White	255	200	80	0xC8 0xFF 0x50	Approximate 2700 K
DU Blue	31	56	100	0x38 0x1F 0x64	University brand colour
Off	0	0	0	0x00 0x00 0x00	LED extinguished

Table 3: WS2812B colour palette and GRB wire encoding

6.3 Algorithm: TIM1 One-Pulse Initialisation for WS2812B

Algorithm A5.1 — TIM1 CH1 One-Pulse Mode Init on PA8

1. Enable GPIOA and TIM1 peripheral clocks.
2. Configure PA8: Alternate Function mode, Push-Pull, Very High Speed, No Pull. Set AFR[1] for PA8 to AF1.
3. Configure TIM1:
 - a. Set One-Pulse Mode bit (OPM) in CR1 — counter auto-stops after one ARR overflow.
 - b. Set PSC = 0 (no prescaling, 180 MHz tick).
 - c. Set ARR to the value computed in Task 1 for one bit period.
 - d. Force register update via EGR.
4. Configure CH1 for PWM Mode 1 with preload (same as Task 4 Algorithm A4.1, steps 6a and 6b).
5. Enable CH1 output (CCER: CC1E = 1).
6. Set the Main Output Enable (MOE) bit in BDTR. **This step is mandatory for TIM1 and TIM8.**

6.4 Algorithm: Sending One Bit

Algorithm A5.2 — WS_SendBit(bit): One Bit Transmission

1. If bit is 1, load CCR1 with the T1H value.
If bit is 0, load CCR1 with the T0H value.
2. Reset the counter (CNT = 0).
3. Start the timer (set CEN in CR1).
4. Busy-wait until the counter stops automatically (one-pulse mode clears CEN when CNT reaches ARR).
Poll CR1 until the CEN bit reads as zero, *or* poll for the Update Interrupt Flag (UIF in SR).
5. Clear the UIF flag before returning.

6.5 Algorithm: Sending a Colour

Algorithm A5.3 — WS2812_SetColor(r, g, b): Send One LED Colour Frame

1. Globally disable interrupts to prevent timing corruption.
2. Transmit the **Green** byte, MSB first:
 - a. For i from 7 down to 0: extract bit i of g using $(g \gg i) \& 1$, then call WS_SendBit() with that value.
3. Repeat step 2 for the **Red** byte.
4. Repeat step 2 for the **Blue** byte.
5. Re-enable interrupts.
6. Drive PA8 low for at least 50 μ s (reset pulse) by temporarily reconfiguring PA8 as a GPIO output, holding it low, waiting with delay_us(55), then restoring AF1.

6.6 Algorithm: HSV Hue Sweep

Algorithm A5.4 — Simple Integer HSV-to-RGB Conversion (S=1, V=1)

1. Accept hue H in the range 0–359 degrees.
2. Compute the sextant: $seg = H/60$ (integer division).
3. Compute the fractional remainder: $frac = H \bmod 60$.
4. Compute the interpolation values (all as unsigned 8-bit integers, 0–255):
 - $q = 255 \times (60 - frac)/60$
 - $t = 255 \times frac/60$
5. Select (R, G, B) based on the sextant (seg = 0: R=255, G=t, B=0; seg = 1: R=q, G=255, B=0; seg = 2: R=0, G=255, B=t; seg = 3: R=0, G=q, B=255; seg = 4: R=t, G=0, B=255; seg = 5: R=255, G=0, B=q).
6. Call WS2812_SetColor(R, G, B).

6.7 Demonstration Requirements

1. Cycle through all 10 colours in Table 4, holding each for one second. For each colour, print over UART:
Colour: <name> R=<r> G=<g> B= GRB=[<g_hex> <r_hex> <b_hex>]
2. Perform a full hue sweep ($H = 0$ to 359, step 3) with a 25 ms pause per step.
3. **Extension (mandatory for full marks):** Drive a chain of 4 WS2812B LEDs with a colour-chase animation: one LED illuminates in red while the rest remain off, and the active LED rotates every 200 ms. Print the active LED index over UART on each step.

6.8 Post-Lab Questions — Task 5

1. Why is it necessary to disable global interrupts (`__disable_irq()`) for the entire bit-stream transmission? Calculate the minimum ISR latency that would corrupt a WS2812B bit at 180 MHz.
2. With four WS2812B LEDs in a chain, each drawing up to 60 mA at full white, calculate the total current draw. The Nucleo's 3.3 V regulator is rated for 300 mA. Is this configuration safe? What protection would you add?
3. A more efficient WS2812B implementation uses DMA to load CCR1 automatically from an array of T0H/T1H values. Describe the data array structure (type, size for 1 LED, for 8 LEDs), the DMA stream and channel required, and the key register settings — without writing any code.

7. Task 6 — Extension: Buzzer, Servo, or Input Capture

Marks: 10 | CO2–CO3 / PO2, PO6 | Bloom C4: Analyse

Choose **one** of the three sub-tasks below. All three follow the same marking breakdown.

Bare-metal implementation, UART-verified 4 marks

HAL/CubeMX implementation 3 marks

Post-lab questions 3 marks

7.1 Option A — Passive Buzzer Tone Generation (TIM4)

Using **TIM4 Channel 1** on PB6 (AF2), generate a square wave at each frequency in Table 5 to play the C-major scale. The duty cycle is fixed at 50%.

Note	Freq (Hz)	PSC	ARR	Your Calculated ARR
C4	261.6	99	3436	_____
D4	293.7	99	3061	_____
E4	329.6	99	2727	_____
F4	349.2	99	2572	_____
G4	392.0	99	2295	_____
A4	440.0	99	2044	_____
B4	493.9	99	1821	_____
C5	523.3	99	1717	_____

Table 4: C-major scale: frequencies and TIM4 register values ($f_{\text{TIM4}} = 90 \text{ MHz}$, $\text{PSC} = 99$)

Demonstration: Play C4–D4–E4–F4–G4–A4–B4–C5, each for 400 ms. Then play a short recognisable melody of your choice. Print each note name, ARR value, and back-calculated actual frequency over UART.

Algorithm guidance:

Algorithm A6.A — Buzzer Note Play

1. Compute ARR for the target frequency using Equation 1 with $\text{PSC} = 99$.
2. Stop the TIM4 counter temporarily.
3. Update ARR and CCR1 ($\text{CCR1} = \text{ARR} / 2$ for 50% duty).
4. Force an Update Event (EGR: $\text{UG} = 1$).
5. Restart the counter.
6. Hold for the desired note duration using `delay_ms()`.

7.2 Option B — Servo Motor Control (TIM2 Channel 1)

Using **TIM2 Channel 1** on PA0 (AF1), generate a 50 Hz PWM signal (20 ms period) to control a standard RC servo motor. Pulse width: 1 ms = 0°, 1.5 ms = 90°, 2 ms = 180°.

Algorithm A6.B — Servo Angle to CCR

1. Determine PSC and ARR for exactly 50 Hz PWM from $f_{\text{TIM2}} = 90 \text{ MHz}$.
2. To convert an angle θ (0 – 180°) to a pulse width t in microseconds:

$$t = 1000 + \frac{\theta}{180} \times 1000 \quad [\mu\text{s}]$$

3. Convert t to a CCR value:

$$CCR = \frac{t}{T_{\text{period}}} \times (ARR + 1) = \frac{t \times f_{\text{timer}}}{10^6}$$

4. Write the computed CCR to TIM2's CCR1 register at runtime.

Demonstration: Sweep the servo from 0° to 180° in 18 steps (10° each), pausing 500 ms per step. Print the target angle, pulse width (μs), and CCR value over UART at each step.

7.3 Option C — Input Capture: Measuring External PWM (TIM5)

Using **TIM5 Channel 1** on PA0 (AF2), configure the timer in *Input Capture* mode. Connect the output of TIM3_CH1 (PA6, from Task 4) to PA0 via a jumper wire and measure its frequency and duty cycle.

Algorithm A6.C — Input Capture Period and Duty Measurement

1. Configure TIM5 CH1 for rising-edge capture and TIM5 CH2 for falling-edge capture on the same input pin (using the indirect mapping: IC2 maps to TI1 input).
2. Start TIM5 as a free-running counter.
3. Capture rising edge timestamp t_{rise1} in CCR1.
4. Capture falling edge timestamp t_{fall} in CCR2.
5. Capture next rising edge t_{rise2} in CCR1.
6. Compute:
 - Period: $T = t_{\text{rise2}} - t_{\text{rise1}}$
 - High time: $t_H = t_{\text{fall}} - t_{\text{rise1}}$
 - Measured frequency: $f = f_{\text{timer}}/T$
 - Measured duty: $D = t_H/T \times 100\%$
7. Print results over UART and compare to the configured TIM3 values.

8. Task 7 — Real-World Applications and SDG Reflection

Marks: 15 | CO3 / PO6 | Bloom A4: Organise

Written response, min. 600 words, submitted in lab report	10 marks
In-lab discussion with instructor (5 minutes)	5 marks

8.1 Written Analysis

Write a structured analysis (minimum 600 words) addressing *all three* of the following parts. Your response should demonstrate understanding of how the embedded techniques from this lab appear in real engineering products.

Part A: Timer-Based Applications in Modern Embedded Systems

Identify and describe *three* real-world products or systems (other than simple LED dimming or desktop clocks) that rely critically on timer peripherals for their correct operation. For each example, explain:

- Which timer function is used (delay, PWM, frequency measurement, or a combination).
- What the timing accuracy requirement is and why it matters.
- What would fail if the timing was 10 % off.

Part B: WS2812B and Smart Lighting in Smart Cities

The WS2812B LED is widely used in smart street lighting and architectural lighting systems. Discuss:

- How PWM duty-cycle control contributes to energy efficiency in municipal lighting infrastructure, with reference to Bangladesh's SDG Goal 7 (Affordable and Clean Energy) and SDG Goal 11 (Sustainable Cities).
- How a network of STM32-based nodes communicating over CAN bus (covered later in the course) could implement adaptive street lighting that responds to ambient light sensors and pedestrian density.
- What firmware design choices (interrupt-driven vs. DMA, low-power sleep modes, watchdog timers) would be essential for a deployment expected to run continuously for 10 years without maintenance.

Part C: Design Challenge

You are asked to design the firmware architecture (not the code, but the architecture) for an **embedded node for a solar-powered IoT greenhouse monitoring system** deployed in rural Bangladesh. The node must:

- Blink an RGB status LED (WS2812B) to indicate system state (green = normal, yellow = warning, red = fault).
- Drive a servo-controlled ventilation flap.
- Measure temperature/humidity sensor response time.

- Transmit sensor readings over UART every 30 seconds.
- Consume as little power as possible between readings.

Describe which STM32F446 timer peripheral you would assign to each task, justify your choice of bare-metal vs. HAL for each, and explain how you would coordinate all tasks without using an RTOS.

Submission Checklist and Final Marks Summary

What to Submit

1. **Lab Notebook / Pre-lab Worksheet** (Task 1): signed by instructor before the session begins.
2. **Source Code** (all tasks, both bare-metal and HAL): submitted as a zipped STM32CubeIDE workspace with clearly named projects (Lab02_T2_BareMetal, Lab02_T2_HAL, etc.).
3. **UART Terminal Logs**: screen-captured or copy-pasted from your serial terminal, one file per task.
4. **Oscilloscope Screenshots** (Tasks 4 and 5 where specified).
5. **Lab Report**: a single PDF containing: post-lab question answers for all tasks, data tables, waveform screenshots, and the Task 7 written analysis.

Final Marks Summary — Lab-02 Assignment

#	Task	CO	Bloom	Max	Obtained
T1	Timer calculations pre-lab worksheet + viva	CO1	C3	10	_____
T2	Delay generation (ms, s, hours)	CO1–2	C3–4	15	_____
T3	Duration measurement and code profiling	CO2	C4	15	_____
T4	PWM generation and LED brightness	CO1–2	C4	15	_____
T5	WS2812B colour mixing and animation	CO2–3	C4–A4	20	_____
T6	Extension: buzzer / servo / input capture	CO2–3	C4	10	_____
T7	SDG reflection and design challenge	CO3	A4	15	_____
Total				100	_____

Academic Integrity Statement

By submitting this assignment, you confirm that the work presented is your own, that no code has been copied from classmates or internet sources without proper attribution, and that you have not used any AI-assisted code generation tool to write the implementation code. **All submitted source files will be checked with similarity detection software.** Violations of the University of Dhaka academic integrity policy will be forwarded to the disciplinary committee.